EECS 4413, Winter 2026: team Project Specifications -- V1

Teams

This is a teamwork project. The size of the team is highly recommended to be a minimum of 2 and a maximum of 4 students per team. Ideally a team should be comprised of 4 members. You should have formed your team and elected a captain, who should register the team info by Feb 20.

Basic functionalities

In this project you are going to create an online e-store for a retail company. Your job involves designing, implementing, testing and deploying a web application that supports Customers and Administrators to interact with the e-store.

Customers can execute several use cases, *including*

- Register (sign up), Sign in, sign out

- List catalogue items
- Sort items by prices, names
- Filter catalog items by categories, brand, model
- View details (description, brand, price etc) of a product

- Add items to shopping cart, edit or remove items from the shopping cart
- "Check out" by providing credit card information and shipping information to purchase the items in the shopping cart

- Maintain profile including personal info as well as purchase history
- etc

Administrators: are the owners of the store; they have access to the following use case

- Maintain sales history.
- Maintain inventory
- Maintain user accounts
- etc

You are welcome to add more features, some examples are given in the Executive Summary part later in this document.

Development and Runtime Frameworks

You will develop a multi-tier web application, with clear separation between front and back-end.

You are allowed to use **any** technology presented in class, but are also encouraged to go beyond the technologies covered in lectures and labs. For example, you can use **Java Spring or Sprint Boots** for back-end content, use **React** or **JS/Angular** for front end, use **Node.js** frameworks for both front and back ends. (Some tutorials on the topics will be provided). You can even use **LAMP/PHP** for the project.

You can also use databases other than MySQL and SQLite used in the class, e.g., use (local or online) PostgreSQL, MariaDB, and even MongoDB etc.

It is expected that each member of the team can contribute to and understand each part of the project and therefore agree on the technologies used in the project.

Design and implementation

It is important that the architecture of your store exhibits:

- Clear separation between front and back-end
- MVC architecture pattern with a clear separation of business logic and presentation. Web API that mediates between front- and back-end,
- DAO design pattern.
- Other architecture and Design patterns when appropriate (Factory, Singleton, Observer etc.)
- Testcases (based on browser, curl, postman) for the back-end
- Good coding style (modularity, comments, readability, etc.)
- Robustness to user inputs

Below are the *SUGGESTED* main components/services of your e-store. It is acceptable to deviate from this if you justify it in the design document of the report. So, your architecture may comprise more components/services than the ones reported below.

Back-end and Model services

A. Data Access

This component mediates between your application's business logic and the data base(s). It should be scalable and configurable.

B. Catalog Component/Service

The Catalog provides the APIs and the implementation to support use cases related to the products sold by the e-store

C. Ordering Component/Service

The Ordering provides the APIs and the implementation of use cases than manage user orders

D. Identity management component/service. Provides the interfaces and the implementation for user registration, log in and log out.

E. Shopping cart component/service. Provides services for managing the shopping basket, including the management of state

Web APIs (this is the set of API exposed by the back-end)

F. Controller/API Gateway

The component/service mediates between the "model" and the "views".

Front-end components

The front end can be built as a collection of dynamic views. Below are the main views, use them as examples:

G. Catalog View

Displays the contents of the store organized by category and by product. The visitor must be able to

1. browse the catalog and see All, By Brand, By Categories products
2. select an item and see the detailed information for that item (description, brand, price etc.).
3. add an individual item to shopping cart.

H. Shopping Cart View

The Shopping Cart Page allows a visitor to review the order

1. view all items in the shopping cart and their information (price, etc.).
2. remove individual items from the shopping cart or increase/decrease the quantity. While doing so, the total bill is updated.
3. button or link that allows user to go back to continue shopping.
4. "Checkout" submit button indicating they wish to purchase the items in the shopping cart.

I. Checkout View

1. If not login yet, either log into their account with a password, or create a new account to check out.
2. for a new account they enter their account name, password, and default billing and shipping information. The new account is submitted to the Order Processing component/service.
3. for existing account, load the saved billing and shipping information. User can choose to use the loaded info, but can also choose to enter a temporary billing and shipping information.
4. "Confirm order" button

Note: You do not need to use a real 3rd party payment service for this project, create a simple payment web service that mimics a real payment. Design a simple algorithm to accept or deny a payment request. E.g., deny every 3rd request of payment, or other dummy algorithms . If the order is approved, you should display order summary." If it is denied, you should display "Credit Card Authorization Failed.", and allow user the enter again or quit.

J. Registration View. Provides the user interface for registering/maintaining an account.

1. log in
2. log out
3. register

K. Administrator/Analytics Page View

The Administrator should be able to

1. view the sales history
2. view inventory history. Can update inventory and add new item to inventory.
3. view user accounts. Can update user information.

Executive summary

You can use any tools you choose. You are free to design the interface and business logic. Listed below are required features (in red), and some examples of extra features (in blue)

Required functionalities:

- Design and implementation:
  - Use MVC architecture pattern with a clear separation of business logic and presentation. Web API that mediates between front- and back-end,
  - Use DAO design pattern.

- Customers can execute several use cases, including
  - Register (sign up), entering customer info (name, credit card info, shipping address etc)
  - Sign in
  - See profile/account page with customer info, purchase history etc.
    - Can update customer info (name, credit card info, shipping address etc)
  - Sign out

  - List catalogue items
    - Items should have images.
  - Filter catalog items by categories, genres, or brand
  - Search product by type, genre, keyword etc..
  - Sort items by prices (sort on ascending and descending prices)
  - Sort items by names (i.e., in the alphabetical order)
  - View details of a product (e.g., by clicking on the product image)
    - Should show details of the product
    - Should also show inventory information (quantity remaining)

  - Add items to shopping cart
  - Edit or remove items from the shopping cart
    - The amount due should change immediately as items are added or removed.
  - Go back to continue shopping. Items in shopping cart were retained.

  Should be able to do these with or without login

  - "Check out" page.
    - If ordering an item with more than its inventory level, give error message and reject the operation (can also do this in the shopping cart page)
    - When an item is checked out, the inventory level should decrease accordingly.

    - If not login yet, prompt to either log into their account with a password, or create a new account to check out.
    - For a new account they enter their account name, password, and default billing and shipping information. The new account is submitted to the Order Processing component/service, where the selected items are retained.
    - If already login with an existing account, load the saved billing (credit card) info and shipping address information.
      - Users can choose to use the loaded info (credit card and shipping address), but can also choose to enter temporary billing and shipping information.

  - In checking out, do payment processing
    - If payment is accepted, get an order summary page. The corresponding product's quantity should be reduced accordingly. Shopping cart depleted. Order added to purchase history.

        - If payment is rejected, should display "Credit Card Authorization Failed.", and allow user the enter again or quit.

- Administrators: they are the owners of the store; they have access to the following basic use cases:
  - Check sales history -- filter by different customers, by product, or by date etc.
    - For each sales order, see order details (user, product, price, quantity etc)
  - Maintain customer account pages with customer info, purchase history etc
    - Can update customer information (credit card, shipping address etc)
  - Maintain inventory
    - Can change inventory quantity of existing products (add or reduce inventory)
  - From inventory management, can add new items with some quality, description.

Extra innovative features

List above are required features. You are welcome to add more features. Extra innovative features could get bonus marks. Listed below are some suggested example features (you are not limited to these features. You are encouraged to be creative in designing your project):

- **Design and implementation**
  - Use design or architecture patterns (in addition to MVC and DAO which are required), e.g,. Singleton, Factory, Observer, Façade etc
  - Has one or more **Web services** that follow REST API (e.g., payment as a separate Web Service).
  - Use some technologies outside class. E.g., React/Angular for front end, Springboot for backend

- **Customer**:
  - After logging out, if there were items in shopping cart (did not checkout), the items are retained when login again
  - Maintain a wish-list that retains items when logging out, and logging again.
  - Read and write reviews on items and rate items using five stars
  - Ask questions to a live chatbot which can answer basic questions
  - System sends confirmation email to customer after checkout.
  - Catalog view shows 'featured/hot/recommended' products, based on sales, to attract customer attention.
  - For items that have low inventory level, marked with some message like 'almost gone..' to attract customer attention
  - For videos, provide trailer view.
  - For books, provide preview
  - For glasses/clothes/wigs etc, provide virtual "try on" on customer uploaded pictures
  - Encrypted login information
  - …. Other creative features…

- **Administrator**:
  - Can generate a summary of sales, preferably with charts for visualization
  - Can run a report on sales with option to save as disk file (e.g. as PDF files).
  - From inventory management, when adding new items, can upload product images
  - Can change the image of an existing product
  - Can Authorize/approve rejected payments
  - …. Other creative features…

Team development and deployment

By the end of the term (**Around Apr3, TBD**)

- Your e-commerce application is expected to be deployed in cloud, including the database, at a public address (e.g., AWS, Google cloud or the web host). That is, the application is runnable by visiting a public address
- Alternatively, on Docker so it can be run by the instructor and TA on other machines.
- (your project is still acceptable if none of the above works, so it runs on local computer. But you will lose about 5-8% marks for deployment)

Main deliverables

- By the end of the term (**Around Apr 3, TBD**), you will present your application.
- By the end of the term, you will submit
  - A project design report (template will be provided soon)
    - should be 10~ 15 pages more details to be added
    - rationale of design choices, design details, class diagrams, database design, contribution of member, challenges, creative features etc
    - more details to be provided.
  - Docker file or link to the application if deployed in the cloud.
    - **One team member should focus more on deployment. Explore use of Docker, or ways to upload to cloud host**. (Other members focus on front end and back end each)
  - Source code of program.
    - If you use cloud GitHub, provide the link to it and give your TA access to it. Otherwise, provide your source files as jar/war file. The TA should see your code and scripts.

Appendix:

Sample SQL schemas for e-store database

Some sample data schemas are attached at the end of this document. You are welcome to use any free database from Cloud (SQL or noSQL). *You need to extend the schema and add more content to the tables.*

-use it as a starting example

- you will need to adapt them for other DBMS
- you will need to extend them as you see fit. For instance, each item may have more attributes, for example, for glasses, may have clour, a shape (e.g., oval, round, square), a size, and a weight (in grams).
- you will need more tables. For example, you may need table for **review, billing_address** etc..

```
/* create Item/product table*/
CREATE TABLE Item(
 itemID  VARCHAR(20)  NOT  NULL  PRIMARY  KEY,
 name VARCHAR(60) NOT NULL,
 description VARCHAR(60) NOT   NULL,
 category VARCHAR(60) NOT NULL,
 brand VARCHAR(60) NOT NULL,
 quantity INT NOT  NULL,
 price INT NOT NULL
 …..
);

/*example insert data into item table*/
INSERT INTO ITEM (ItemID, name, description, category, brand, price, quantity) VALUES ('b001',
'Little Prince', 'a book for all ages', 'book', 'penguin', 20, 100);
INSERT INTO ITEM (ItemID, name, description, category, brand, price, quantity) VALUES ('c001',
'iPad', 'a portable device for personal use', 'computer', 'Apple', 500, 100);
INSERT INTO ITEM (ItemID, name, description, category, brand, price, quantity) VALUES
('d001', 'laptop', 'a laptop for personal use', 'Dell',  1500, 100);

/*create an address table*/
CREATE TABLE Customer (
 id   INT NOT NULL,
 firstName VARCHAR(100) NOT NULL,
 lastName VARCHAR(20) NOT NULL,
 addressID INT  NOT NULL,
 …..

 PRIMARY KEY(id)
 FOREIGN KEY (addressID) REFERENCES Address (id)
);

/*create an address table*/
CREATE TABLE Address ( id    INT NOT NULL,
 street VARCHAR(100) NOT NULL,
 province VARCHAR(20) NOT NULL,
 country VARCHAR(20) NOT NULL,
```

```
 zip     VARCHAR(20) NOT NULL,
 phone   VARCHAR(20),
 …..
 PRIMARY KEY(id),
);

/*populate the address table*/

INSERT INTO Address (id, street, province, country, zip, phone) VALUES (1, '567 Yonge St',
'ON', 'Canada', 'K1E 6T5' ,'647-123-4567');
INSERT INTO Address (id, street, province, country, zip, phone) VALUES (2, '945 Avenue
rd', 'ON', 'Canada', 'M1C 6K5' ,'416-123-8569');
INSERT INTO Address (id, street, province, country, zip, phone) VALUES (3, '189 Keele
St.', 'ON', 'Canada', 'K3C 9T5' ,'416-123-9568');


/* create Purchase Order (PO) table */  used for viewing sales history

/* Purchase Order
CREATE TABLE PO
 ( id INT NOT NULL,
 customerID VARCHAR(20) NOT NULL,
 itemID  VARCHAR(20) NOT NULL,
 dateOfPurchase  VARCHAR(20) NOT NULL,
 ….

 PRIMARY KEY(id),
 FOREIGN KEY (customerID) REFERENCES Customer (id)
 FOREIGN KEY (itemID) REFERENCES Item (id)

);
```